

第 4 组单元测试报告

模块：工具注册 + OS Skills (ToolRegistry / SkillLibrary / Mock)

- 测试环境：Windows 11 Pro + Python 3.13.5 (Ubuntu 20.04 亦通过 CI 级兼容验证)
- 测试框架：pytest
- 报告日期：2026-09-11 (最终定稿)
- 板块基调：cores 核心模块 + mock 全量，590 项测试全绿

一、测试概况

项目	数值
测试文件数	25
测试用例数	590
通过 / 失败	590 / 0
退出码	0
覆盖模块	ToolRegistry、SkillLibrary、OSServiceAPI、Mock 全模块、Agent、LLM 客户端、会话、安全、视觉、邮件、编程辅助、浏览器视觉定位 (UITarget / browser_inspect / browser_find) 等

本报告为最终交付态测试结果（590 项全绿，新增 tests/test_agent_obs.py 观察生命周期 7 项），历史里程碑仍保留在 第4组项目进度跟踪.md 的可复现测试记录中。测试文件数现为 26。

二、设计书功能要求 → 测试覆盖映射

设计书要求	对应测试	用例数
ToolRegistry 统一调用接口	register/call/list_available_tools/unregister 正常与异常	test_tool_registry.py 及派生测试
SkillLibrary 系统管理操作	技能列表/各技能运行/动态注册	test_skills.py、test_agent_skills.py
工具签名（可选）	签名生成与完整性校验	ToolRegistry 签名测试
调用统计（必做）	get_stats：空统计、调用后统计、清空	统计测试
参考 MCP 协议	优雅接口 + Schema 生成	接口/清单测试

设计书要求	对应测试	用例数
Mock（跨组联调）	<code>test_mock.py</code> ：61 工具全量 + MockSkill + MockAPI	<code>test_mock.py</code> 系列
系统协调/安全/RAG（第 5 组）	<code>test_coordinator.py</code> ：SystemCoordinator 编排 + SecuritySandbox 四层 + RAG 检索 + 审计 + 降级 Mock	<code>test_coordinator.py</code> 、 <code>test_coordinator_agent.py</code>
危险指令硬拦截	<code>TestSandboxGuard</code> ：rm/format/shutdown/dd/mkfs/fdisk/Stop-Computer 等 10 风险令直接拒绝 + 正常命令不受影响	<code>test_agent.py</code>
浏览器视觉定位（DOM-SOM-坐标）	<code>test_uitarget.py</code> （UITarget：ref/视觉编号/bbox/center 构造与摘要）、 <code>test_browser_som.py</code> （交互优先级排序 + 带编号覆盖层图 + InspectSession 生命周期 + som:N→ref→真实落点 + SoMStaleError 失效判定 + <code>browser_find</code> 局部搜索）	<code>test_uitarget.py</code> 、 <code>test_browser_som.py</code>

三、重点测试覆盖明细

3.1 ToolRegistry（core/tool_registry.py）

- 注册管理：注册、别名（`move_file == move_path`）、注销、未注册工具调用 → `{success:false,error}`
- 文件/文件夹操作：`copy/move/rename/delete`、`create/delete/list`、`sort_directory`、`index_files`、`manage_archive`
- 文本/批量/查找：`read/write/edit_file`、`batch_copy/move`、`search_files`、`search_in_files`、`find_*`
- 磁盘/图片/系统：`disk_usage`、`get_image_metadata`、`current_directory`、`run_command`（正常与失败）
- 异常处理：缺少参数、非法参数、工具抛异常 → 归一为 `{success:false,error}`
- 调用统计：`get_stats()` 空统计、累计次数（by_tool 排序）、结果记录、清空
- 工具签名：注册后生成 SHA256 签名、`list_tool_signatures`、完整性校验匹配/不匹配

3.2 SkillLibrary（core/skill_library.py，26 技能）

- 信息查询类：`system_info` 返回结构化系统信息
- 文件整理类：`organize_downloads`（演示场景 6）、`smart_organize`、`cleanup_temp`、`cleanup_by_type`、`deduplicate_files`
- 回收站/备份类：`trash_file`、`restore_file`、`empty_trash`、`safe_delete`、`backup_file`、`restore_backup`
- 搜索/索引类：`search_file`、`rebuild_index`、`query_index`、`duplicate_finder`
- 动态注册：运行时注册自定义技能并调用

3.3 Mock（mock/mock_tools.py，61 工具全量）

- `MockToolRegistry` 与正式版同名/同参/同返回，内存实现不改磁盘
- `MockSkillLibrary`、`MockOSServiceAPI` 三件套全覆盖
- 异常路径：未知工具/未知技能/缺参

3.4 Agent 端到端（工具驱动）

- function-calling 驱动的多步工具调用（真实 LLM 联调见《联调与集成报告》）
- 工具结果截断（超长输出省 token）、循环上限、错误回喂重试

四、运行方式

```
# 全量（推荐，最终验收）
python -m pytest -q

# 定向
python -m pytest tests/test_tool_registry.py -q
python -m pytest tests/test_skills.py -q
python -m pytest tests/test_mock.py -q
```

五、结论

第 4 组工具服务层功能完整、测试充分，590 项测试全绿，满足设计书“模块独立运行 + 单元测试通过”的验收要求；Mock 全量可用，支撑跨组联调与整体集成。

六、本里程碑新增能力与回归（2026-09-12）

本轮围绕“浏览器操作可靠 + 过程可观测”新增以下能力，全部经 `pytest` 回归（583 项仍全绿）：

能力	实现位置	说明与测试入口
浏览器会话硬门控	<code>tool_registry.browser_launch</code> + <code>_BROWSER_SESSION_LOCKED</code>	浏览器任务期间拦截 <code>click_at/send_text/send_hotkey/activate_window</code> ，杜绝网页任务误切桌面工具（上轮豆包失败主因）；读操作放行； <code>browser_close</code> 解除。
窗口前置	<code>tool_registry._bring_browser_to_front</code> + <code>browser_bring_to_front</code>	<code>browser_launch/browser_navigate</code> 后自动 <code>SetForegroundWindow</code> ，用户可见操作过程；策略为尽力而为（Wayland 下需 Xorg）。
SPA 恢复	<code>browser_refresh</code> （ <code>Page.reload</code> + 回读 URL/title）	页面空白/加载异常时，提示 <code>wait→refresh→re-snapshot</code> ，不切桌面。
<code>browser_wait</code> 类型兜底	<code>tool_registry.browser_wait</code> / <code>browser_controller.wait_for</code>	<code>timeout</code> 内部 <code>float()</code> ，杜绝 <code>float+str</code> 崩溃。
幂等复用	<code>browser_launch</code> 存活探测	同一持久档案不二次 <code>spawn</code> 抢 9222 端口。
持久登录档案	<code>config.yaml browser.user_data_dir</code>	专用浏览器窗口，登录态落 <code>%LOCALAPPDATA%/miniyu_edge</code> ， <code>browser_close</code> 不清除。

能力	实现位置	说明与测试入口
跨平台修补	<code>browser_controller</code> <code>start_new_session</code> + <code>killpg</code> 、 <code>_find_chrome</code> 扩充	Linux 整树杀进程、chromium/snap 等路径探测。
Token 用量记录	<code>agent._record_llm_usage</code> / <code>token_usage</code> / <code>_usage_note</code>	每步 <code>prompt/completion/total</code> + 图片数 + 想法 + 时间，写 <code>run_log_*.jsonl</code> ；最终回复附本轮/本对话/预算三栏； <code>token_usage</code> 工具可查分步。
过程时间线	<code>agent._log_action</code> （经 <code>_execute_one</code> 包装）	每次工具调用记时间/名称/参数(脱敏)/成功与否/耗时，与 LLM 步合并为完整执行时间线。

| Mock 侧工具数为 61（不含新增的工具）；真实 ToolRegistry 为 63，Agent 可见工具集为 71。回归方式：
`python -m pytest tests/ -q`。 |

追加（P1 成本优化，2026-09-11 晚，token 性能归因后落地）：

能力	实现位置	说明
Observation 生命周期（历史图片裁剪）	<code>agent._trim_history_images</code> + <code>_MAX_HISTORY_IMAGES=2</code>	发往 LLM 的历史最多保留最近 2 张观测截图，更旧的图片从载荷剥离、文字保留；对话持久记录/产物目录仍存全量。化解“旧图逐轮重发”这一 token 膨胀主因。
结构化优于自动截图（P1-B）	<code>agent._browser_action_needs_visual</code> + 两处截图门控	成功的 <code>snapshot/find/type/wait/read</code> 不再自动截图；仅 <code>launch/navigate</code> 、 <code>click</code> 引起页面/URL 变化、以及失败诊断时才截。桌面全局操作保留保底截图。
结构化读文字守则	System Prompt 第 14 条	读回复/数字/状态一律走 <code>browser_read_text</code> / <code>snapshot</code> ，不靠截图视觉；并约束避免“先试 A 不行再试 B”式冗长自我复述以减少思考 token。
新增回归测试	<code>tests/test_agent_obs.py</code> （7 项）	覆盖历史图片裁剪边界、 <code>cap=0</code> 关闭、视觉判定（结构化不截 / <code>launch</code> / <code>click</code> 状态变化 / 桌面保底）。

实测依据（豆包任务 `run_log_*.jsonl`）：单张观测图=2466 token 且随图片数严格线性累加，末轮 6 张图占输入约 50%；旧图剔除 + 成功不自动截图两招叠加，可在不损失功能前提下把简单网页任务的图片数从 6 压到接近 0~1。